Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

Scan ID 6	Date May 19, 2026
Organization lukso-network	Repository lsp-smart-contracts
Branch main	Commit Hash 446155ae...0e6d59dc

Contracts in scope

packages/lsp7-contracts/contracts/presets/LSP7CustomizableToken.sol

packages/lsp7-contracts/contracts/presets/LSP7CustomizableTokenConstants.sol

packages/lsp7-contracts/contracts/presets/LSP7CustomizableTokenInit.sol

packages/lsp7-contracts/contracts/presets/LSP7Mintable.sol

packages/lsp7-contracts/contracts/presets/LSP7MintableInit.sol

packages/lsp7-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedAbstract.sol

packages/lsp7-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedConstants.sol

packages/lsp7-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedErrors.sol

packages/lsp7-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedInitAbstract.sol

packages/lsp7-contracts/contracts/extensions/AccessControlExtended/IAccessControlExtended.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedBalance/ILSP7CappedBalance.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedBalance/LSP7CappedBalanceAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedBalance/LSP7CappedBalanceErrors.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedBalance/LSP7CappedBalanceInitAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedSupply/ILSP7CappedSupply.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedSupply/LSP7CappedSupplyAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedSupply/LSP7CappedSupplyErrors.sol

packages/lsp7-contracts/contracts/extensions/LSP7CappedSupply/LSP7CappedSupplyInitAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7Mintable/ILSP7Mintable.sol

packages/lsp7-contracts/contracts/extensions/LSP7Mintable/LSP7MintableAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7Mintable/LSP7MintableErrors.sol

packages/lsp7-contracts/contracts/extensions/LSP7Mintable/LSP7MintableInitAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7NonTransferable/ILSP7NonTransferable.sol

packages/lsp7-contracts/contracts/extensions/LSP7NonTransferable/LSP7NonTransferableAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7NonTransferable/LSP7NonTransferableErrors.sol

packages/lsp7-contracts/contracts/extensions/LSP7NonTransferable/LSP7NonTransferableInitAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/ILSP7Revokable.sol

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/LSP7RevokableAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/LSP7RevokableErrors.sol

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/LSP7RevokableInitAbstract.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableToken.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableTokenConstants.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableTokenInit.sol

packages/lsp8-contracts/contracts/presets/LSP8Mintable.sol

packages/lsp8-contracts/contracts/presets/LSP8MintableInit.sol

packages/lsp8-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedAbstract.sol

packages/lsp8-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedConstants.sol

packages/lsp8-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedErrors.sol

packages/lsp8-contracts/contracts/extensions/AccessControlExtended/AccessControlExtendedInitAbstract.sol

packages/lsp8-contracts/contracts/extensions/AccessControlExtended/IAccessControlExtended.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/ILSP8CappedBalance.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/LSP8CappedBalanceAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/LSP8CappedBalanceErrors.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/LSP8CappedBalanceInitAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedSupply/ILSP8CappedSupply.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedSupply/LSP8CappedSupplyAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedSupply/LSP8CappedSupplyErrors.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedSupply/LSP8CappedSupplyInitAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Mintable/ILSP8Mintable.sol

packages/lsp8-contracts/contracts/extensions/LSP8Mintable/LSP8MintableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Mintable/LSP8MintableErrors.sol

packages/lsp8-contracts/contracts/extensions/LSP8Mintable/LSP8MintableInitAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/ILSP8NonTransferable.sol

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/LSP8NonTransferableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/LSP8NonTransferableErrors.sol

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/LSP8NonTransferableInitAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/ILSP8Revokable.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableErrors.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableInitAbstract.sol

Code Statistics

Findings	Contracts Scanned	Lines of Code
7	60	6187

Findings Summary



Total Findings

- High Risk (0)
- Medium Risk (1)
- Low Risk (3)
- Info (3)
- Best Practices (0)

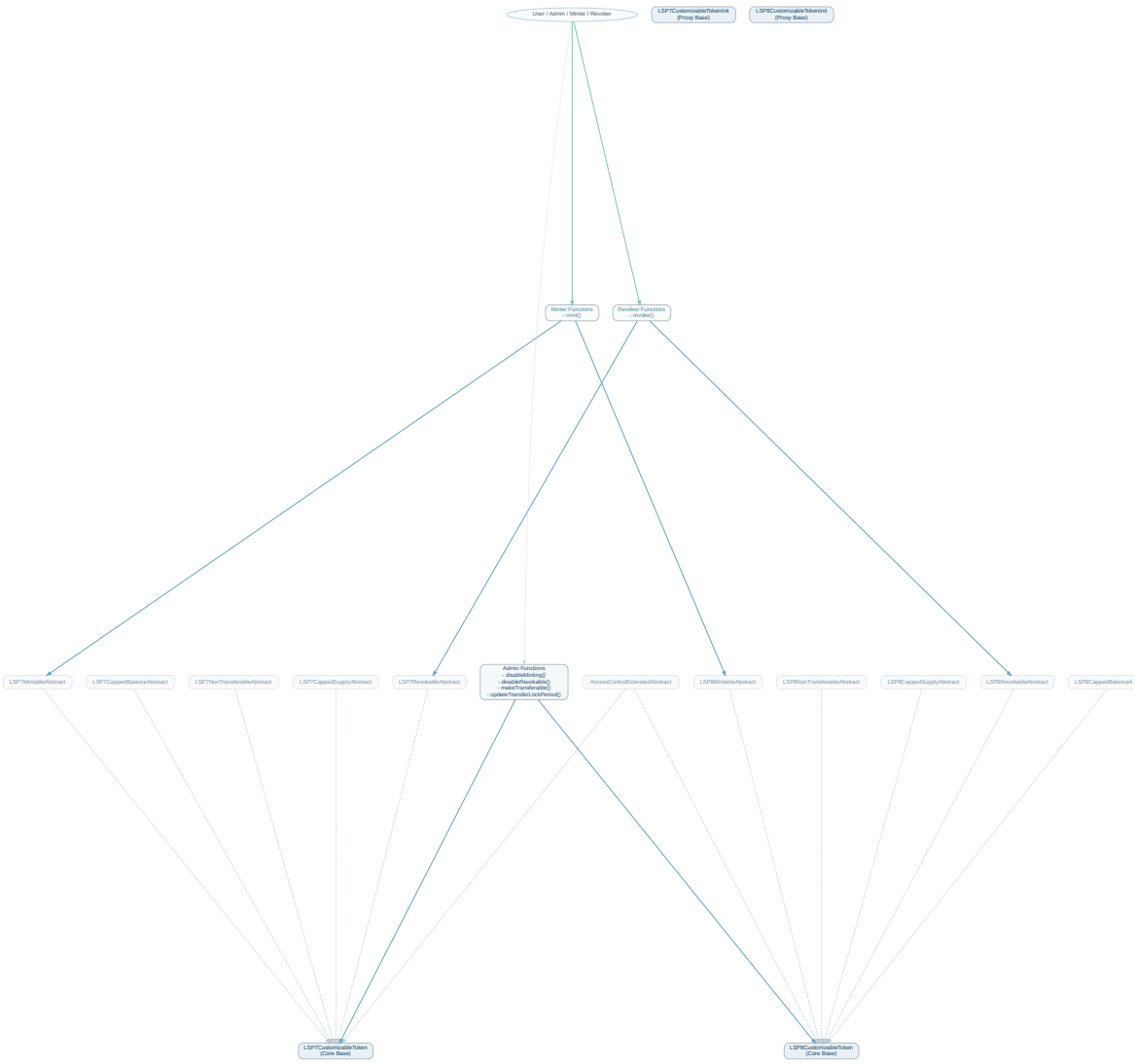
Code Summary

The protocol provides highly customizable and extensible token implementations based on the LSP7 (Fungible Token) and LSP8 (Identifiable Digital Asset / NFT) standards. These customizable tokens consolidate a variety of modular features that can be configured upon deployment or toggled dynamically. Key capabilities include minting with role-based access control, token burnability, maximum supply caps, per-address balance caps, non-transferability with time-based lock periods and bypass roles, as well as revocability mechanisms allowing authorized actors to reclaim tokens. The protocol utilizes an extended access control system with reverse lookups to manage permissions such as minters, revokers, and feature-exemption roles.

Main Entry Points and Actors

- `mint`: Allows authorized accounts to mint new tokens or NFTs to a specified address. Actor: Minter (`MINTER_ROLE`).
- `revoke`: Reclaims tokens or NFTs from a specified holder and transfers them to the contract owner or another authorized revoker. Actor: Revoker (`REVOKER_ROLE`).
- `grantRole`: Grants a specific role to a given account. Actor: Role Admin.
- `revokeRole`: Removes a specific role from a given account. Actor: Role Admin.
- `renounceRole`: Allows an account to voluntarily relinquish a role it currently holds. Actor: Role Holder.
- `setRoleAdmin`: Configures the admin role that manages a specific role. Actor: Default Admin (`DEFAULT_ADMIN_ROLE`).

Code Diagram



1 of 7 Findings

packages/lsp7-contracts/contracts/presets/LSP7CustomizableToken.sol

packages/lsp7-contracts/contracts/presets/LSP7CustomizableTokenInit.sol

Revocation to owner is blocked by the balance cap, contradicting expected behavior and allowing DoS on revokes

Medium Risk

The protocol documentation and design choices explicitly state that token revocation should not be blocked by the balance cap if the owner has lost the `UNCAPPED_BALANCE_ROLE` (`"revoke-to-owner not blocked by balance cap if owner lost UNCAPPED_BALANCE_ROLE"`). When an authorized revoker reclaims tokens and sends them to the contract `owner()`, the transaction is expected to bypass the balance cap.

However, while `LSP7CustomizableToken` successfully overrides `_nonTransferableCheck` to bypass transfer locks when `msg.sig == this.revoke.selector`, it fails to implement a similar bypass for the balance cap check within the `_beforeTokenTransfer` hook. The `_beforeTokenTransfer` function simply delegates to `super._beforeTokenTransfer(...)`, which sequentially triggers `LSP7CappedBalanceAbstract._beforeTokenTransfer`. This parent function unconditionally enforces the `tokenBalanceCap` on the recipient unless they possess the `UNCAPPED_BALANCE_ROLE`.

If the owner does not have the `UNCAPPED_BALANCE_ROLE`, any `revoke` operation attempting to send reclaimed tokens to the owner will revert if the owner has reached the balance cap. A malicious user can exploit this by artificially filling the owner's balance cap (e.g., by transferring small amounts of tokens to the owner) to effectively create a Denial of Service (DoS), preventing authorized revokers from reclaiming the attacker's tokens to the owner.

Severity Note:
- Operationally, revokes are expected/required to target owner(), not merely any revoker address.
- Transfers to owner are possible at some times (non-transferable windows or roles do not permanently block attacker transfers).
- Admins are not continuously holding UNCAPPED_BALANCE_ROLE on owner by design/policy.

2 of 7 Findings

packages/lsp8-contracts/contracts/presets/LSP8CustomizableToken.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableTokenInit.sol

`revoke` operations are incorrectly blocked by the balance cap if the recipient lacks the `UNCAPPED_BALANCE_ROLE`

Low Risk

The protocol documentation and Developer Q&A explicitly state an invariant regarding the `revoke` feature: *"revoke-to-owner not blocked by balance cap if owner lost UNCAPPEDBALANCEROLE"*. This means that when an authorized revoker reclaims a token and sends it to the owner, the transaction should succeed even if the owner's balance has reached the `tokenBalanceCap` and they do not possess the `UNCAPPED_BALANCE_ROLE`.

However, the current implementation fails to enforce this bypass. In both `LSP8CustomizableToken` and `LSP8CustomizableTokenInit`, the developers correctly implemented a bypass for the non-transferability lock by overriding the `_nonTransferableCheck` function. When `msg.sig == this.revoke.selector` and the recipient is valid, it returns early.

Conversely, there is no such bypass implemented for the balance cap check. The `_beforeTokenTransfer` function simply delegates to `super._beforeTokenTransfer(...)`, which systematically executes the checks inside `LSP8CappedBalanceAbstract`. Because the `owner()` is only granted the `DEFAULT_ADMIN_ROLE` during initialization and does not automatically possess the `UNCAPPED_BALANCE_ROLE`, their token balance is strictly subject to the `tokenBalanceCap`.

If the owner's balance has reached the cap, any `revoke` operation targeting the owner will revert within the `LSP8CappedBalanceAbstract` logic. This strictly violates the developer's stated invariant and creates a Denial of Service (DoS) for the `revoke` functionality, preventing authorized revokers from successfully reclaiming tokens.

3 of 7 Findings

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/LSP7RevokableAbstract.sol

packages/lsp7-contracts/contracts/extensions/LSP7Revokable/LSP7RevokableInitAbstract.sol

packages/lsp7-contracts/contracts/presets/LSP7CustomizableToken.sol

packages/lsp7-contracts/contracts/presets/LSP7CustomizableTokenInit.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableInitAbstract.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableToken.sol

packages/lsp8-contracts/contracts/presets/LSP8CustomizableTokenInit.sol

Ownership transfer can be permanently DoSed by bloating the REVOKER_ROLE member set

Low Risk

The revokable extensions perform an unbounded full-set cleanup of `REVOKER_ROLE` during `_transferOwnership`. This makes ownership transfer and ownership renunciation scale linearly with the total number of revokers. Once the set becomes large enough, `transferOwnership(...)` and `renounceOwnership()` can exceed the block gas limit and become uncallable.

The affected logic is:

```
function _transferOwnership(
    address newOwner
) internal virtual override(AccessControlExtendedAbstract, Ownable) {
    _setRoleAdmin(REVOKER_ROLE, DEFAULT_ADMIN_ROLE);
    super._transferOwnership(newOwner);

    address[] memory revokers = getRoleMembers(REVOKER_ROLE);

    for (uint256 ii = 0; ii < revokers.length; ++ii) {
        address revoker = revokers[ii];
        if (revoker == newOwner) continue;

        _revokeRole(REVOKER_ROLE, revoker);
    }
}
```

The init variant contains the same pattern.

This is not just an efficiency problem. `REVOKER_ROLE` membership is externally configurable through the access-control system, and `setRoleAdmin(...)` allows the default admin to delegate administration of that role. If `REVOKER_ROLE` is ever made self-administered, or administered by another non-owner role, an authorized revoker-side actor can add arbitrarily many revokers and then make ownership handover impossible. Even without a malicious configuration, a token that legitimately accumulates a large compliance/recovery revoker set can lose the ability to rotate ownership, transfer upgrade authority, or renounce ownership because the cleanup is forced into a single transaction.

Because ownership is the root administrative capability for these presets, making `_transferOwnership` unexecutable can freeze governance handover and emergency admin recovery paths.

Severity Note:
- Owner (or governance) controls DEFAULT_ADMIN_ROLE and can call setRoleAdmin/revokeRole.
- There exists an external revokeRole flow to batch-remove revokers before attempting ownership transfer.
- Attackers cannot prevent the admin from resetting the REVOKER_ROLE admin to DEFAULT_ADMIN_ROLE.

4 of 7 Findings

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/LSP8CappedBalanceAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8CappedBalance/LSP8CappedBalanceInitAbstract.sol

LSP8CappedBalance incorrect balance calculation blocks valid self-transfers when at the balance cap

Low Risk

In the `LSP8CappedBalance` extension, the `_tokenBalanceCapCheck` function is invoked during `_beforeTokenTransfer` to ensure a recipient does not exceed the `tokenBalanceCap()`. The function calculates the expected new balance by adding 1 to the recipient's current balance: `require((balanceOf(to) + 1) <= tokenBalanceCap(), ...)`.

However, if a user performs a self-transfer (i.e., `from == to`), their balance will not increase after the transfer is complete. The current check overestimates the post-transfer balance by 1. If a user holds exactly the maximum allowed tokens (`tokenBalanceCap()`) and attempts to transfer a token to themselves, the check evaluates to `cap + 1 <= cap`, which incorrectly reverts the transaction. This unnecessarily blocks valid self-transfers (which are completely legal in LSP8/ERC721 and may be used to trigger receiver hooks or reset states) for users who are at the balance limit.

Severity Note:
- Self-transfers are not relied upon for mission-critical flows (e.g., no protocol fund recovery depends on self-transfer).
- The cap is reachable in practice but not universally for all users.

5 of 7 Findings packages/lsp8-contracts/contracts/presets/LSP8CustomizableTokenInit.sol

Initializer-time LSP1 reentrancy can bypass the configured supply cap during `_initialMint`

Info

`LSP8CustomizableTokenInit.__LSP8CustomizableToken_init` finishes by calling `_initialMint`, which performs only a one-time pre-check:

```
uint256 configuredTokenSupplyCap = LSP8CappedSupplyInitAbstract.tokenSupplyCap();
bool exceedsSupplyCap = initialMintTokenIds.length > configuredTokenSupplyCap;
require(configuredTokenSupplyCap == 0 || !exceedsSupplyCap,
LSP8CappedSupplyCannotMintOverCap());
```

It then loops and mints each tokenId through `LSP8IdentifiableDigitalAssetInitAbstract._mint(...)` directly:

```
for (uint256 ii = 0; ii < initialMintTokenIds.length; ++ii) {
    LSP8IdentifiableDigitalAssetInitAbstract._mint(
        to,
        initialMintTokenIds[ii],
        true,
        ""
    );
}
```

This direct base call intentionally bypasses the customizable token's overridden `_mint` path, so the per-mint capped-supply check in `LSP8CappedSupplyInitAbstract._mint(...)` is not executed for the initial-mint loop. In the proxy variant, this becomes exploitable because initialization happens on a fully deployed proxy and the mint path can invoke LSP1 recipient hooks on `newOwner_` if it is a contract receiver. At that point, the receiver already owns `DEFAULT_ADMIN_ROLE` because `__AccessControlExtended_init_unchained()` has run earlier in the same initializer.

A malicious `newOwner_` contract can therefore reenter during the first initial mint, grant itself `MINTER_ROLE`, and call the public mint function to consume the remaining supply headroom up to the configured cap. When control returns to `_initialMint`, the loop continues minting the rest of `initialMintTokenIds` through the uncapped base `_mint` path. The final total supply can therefore exceed `tokenSupplyCap`, permanently breaking the collection's supply invariant.

The issue is specific to the init/proxy preset because constructor-time deployments cannot be reentered through the token contract before runtime code is installed, while proxy initialization can be reentered normally through external hooks.

Severity Note:
- LSP8 minting to a contract receiver triggers the LSP1/universalReceiver hook enabling reentrancy.
- __AccessControlExtended_init_unchained assigns DEFAULT_ADMIN_ROLE aligned with owner set by _initialize, before _initialMint runs.
- Public mint and grantRole are callable during initialize and not blocked by the Initializable guard.

6 of 7 Findings

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/LSP8NonTransferableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8NonTransferable/LSP8NonTransferableInitAbstract.sol

LSP8NonTransferable updateTransferLockPeriod fails to revert if lock period has already started

Info

The NatSpec documentation for the `updateTransferLockPeriod` function in the `ILSP8NonTransferable` interface explicitly states:

```
@dev Can only be called by the contract owner. Reverts if the current lock period has already started or
```

However, the implementation of `updateTransferLockPeriod` in `LSP8NonTransferableAbstract` and `LSP8NonTransferableInitAbstract` completely lacks this validation. It only checks if the transfer lock feature is enabled and if `newTransferLockEnd >= newTransferLockStart`.

As a result, the contract owner can arbitrarily change the lock period at any time, even after the tokens have been locked and the period has started. This allows the owner to unexpectedly extend the lock duration indefinitely while the tokens are actively locked, violating the explicit guarantees made to users regarding the immutability of an active lock period.

Severity Note:
- Integrators and users rely on the NatSpec guarantee that an active or ended lock period is immutable.
- Holders without NON_TRANSFERABLE_BYPASS_ROLE are subject to lock enforcement.

7 of 7 Findings

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableAbstract.sol

packages/lsp8-contracts/contracts/extensions/LSP8Revokable/LSP8RevokableInitAbstract.sol

`transferOwnership` can be gas-DoSed by inflating the `REVOKER_ROLE` member set

Info

Ownership transfer in the revokable extension performs an unbounded cleanup loop over every `REVOKER_ROLE` member:

```
address[] memory revokers = getRoleMembers(REVOKER_ROLE); for (uint256 ii = 0; ii < revokers.length; ++i
```

The role set has no hard cap, and the protocol explicitly exposes `grantRole`, `revokeRole`, and `setRoleAdmin`. This means a malicious or compromised admin of `REVOKER_ROLE` can add enough members to make `transferOwnership` exceed the block gas limit. Once the set is large enough, any ownership handover attempt reverts before completion.

This is not just an efficiency issue: it can block key rotation, incident response, or governance migration for the entire token contract. The risk is amplified by the fact that `REVOKER_ROLE` admin can be delegated away from the owner via `setRoleAdmin`, so a non-owner privileged actor can create the bloated role set and hold ownership transfer hostage.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.]
No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.